

CURT Backend Assessment: Professional Engineering Roadmap

This roadmap follows a strict **Design-First** methodology. It separates the project into system architecture planning, database infrastructure, and iterative API development.

Do not write code until Phase 5. Do not proceed to a new phase until the current one is fully verified.

Part 1: System Design & Architecture (The "Paperwork")

Phase 1: Entity-Relationship Diagram (ERD) Creation

- **Tool:** draw.io / diagrams.net
- **Goal:** Visually map the entities and their relationships.
- **Action:** Draw boxes for `Users`, `Projects`, `Tasks`, and `Project_Members` (junction table for many-to-many). Draw connecting lines showing 1:M and M:M relationships.
- **How to Verify:** Trace the user journey on the diagram. Can a User belong to multiple Projects? Can a Project have multiple Tasks? Yes.

Phase 2: Schema & Table Mapping (Visual/Document)

- **Tool:** Notion / Word / Excel / draw.io
- **Goal:** Define the exact columns for each table based on the ERD.
- **Action:** Write down every table name and its columns. Example: `Tasks` -> id, title, description, status, priority, project_id, assigned_to.
- **How to Verify:** Cross-reference with the CURT PDF requirements to ensure no required fields (like status or priority) are missing.

Phase 3: Data Dictionary & Constraints Definition

- **Tool:** Word Document / Markdown File (`database-design.md`)
- **Goal:** Define strict data types, enums, and database-level rules.
- **Action:**
 - Define Enums: Status (`To Do` , `In progress` , `Done`), Priority (`Low` , `Medium` , `High`).
 - Define Constraints: `email` must be UNIQUE.

- Define Triggers/Actions: `ON DELETE CASCADE` for tasks when a project is deleted.
- **How to Verify:** Peer-review the document. Ask: "If a project is deleted, do orphan tasks remain?"
The document should explicitly say "No, CASCADE handles it."

Phase 4: API Endpoint Mapping

- **Tool:** Markdown File (`api-design.md`) or Postman Drafts
 - **Goal:** Plan the exact Request/Response payloads for every route.
 - **Action:** Document routes (e.g., `POST /register`, `GET /tasks?status=done`). Define what the JSON body should look like and what the success/error JSON response will be.
 - **How to Verify:** Ensure every requirement from the PDF has a corresponding endpoint mapped out on paper.
-

Part 2: Infrastructure & Database Setup (Code Begins)

Phase 5: Server Initialization & Folder Structure

- **Files:** `package.json`, `tsconfig.json`, `src/index.ts`
- **Goal:** Initialize TypeScript, Express, and mirror the exact folder structure required.
- **Action:** Run `npm init -y`, install Express/TS. Create all folders (`config`, `controllers`, `middleware`, `routes`, `scripts`, `utils`, `validators`). Make a basic "Hello World" GET route.
- **How to Test:** Run `npm run dev`. Hit `http://localhost:3000` to see the success message.

Phase 6: Supabase Connection & Drizzle Config

- **Files:** `.env`, `drizzle.config.ts`, `src/config/db.ts`
- **Goal:** Connect your app to your Supabase PostgreSQL instance.
- **Action:** Get the Postgres connection string from Supabase. Put it in `.env`. Initialize the `postgres.js` or `pg` driver and Drizzle client in `db.ts`. Configure `drizzle.config.ts` to point to your `drizzle/schema.ts` file.
- **How to Test:** Add a `console.log("Supabase Connected")` in `db.ts`, import it into `index.ts`, and check the terminal.

Phase 7: Drizzle Schema Translation

- **Files:** `drizzle/schema.ts`

- **Goal:** Translate your Phase 3 Data Dictionary into TypeScript/Drizzle code.
- **Action:** Write the table definitions using Drizzle's pg-core (`pgTable` , `varchar` , `uuid` , `timestamp`). Apply the exact constraints (`notNull` , `unique` , `references (() => table.id, { onDelete: 'cascade' })`) planned in Phase 3.
- **How to Test:** Run `npx drizzle-kit generate` to check for syntax errors. No DB changes happen yet.

Phase 8: Migrations, Drop & Seed Scripts

- **Files:** `src/scripts/drop.ts` , `src/scripts/seed.ts`
 - **Goal:** Apply the schema to Supabase and build automated data population.
 - **Action:**
 - Run `npx drizzle-kit push` to create tables in Supabase.
 - Write a `drop.ts` script using `TRUNCATE` or Drizzle deletes.
 - Write a `seed.ts` script to insert 3 users, 2 projects, and 5 tasks.
 - **How to Test:** Check Supabase Table Editor to verify tables exist. Run drop, then seed. Verify Supabase contains the mock data.
-

Part 3: Core Architecture & Utilities

Phase 9: Global Error Handling

- **Files:** `src/utils/appError.ts` , `src/middleware/errorHandler.ts`
- **Goal:** Standardize API error responses (No raw stack traces).
- **Action:** Create a custom `AppError` class. Create a middleware that intercepts all errors and returns a clean `{ status, message }` JSON.
- **How to Test:** Manually `throw new AppError('Test', 400)` in `index.ts` . Verify Postman shows the clean JSON.

Phase 10: Async Wrapper

- **Files:** `src/utils/catchAsync.ts`
- **Goal:** Eliminate `try/catch` blocks in every controller.
- **Action:** Write a wrapper function that catches unhandled promise rejections and passes them to `next()` .
- **How to Test:** Wrap a test route that throws an async error. Ensure Phase 9's handler catches it.

Phase 11: Zod Validation Setup

- **Files:** `src/middleware/validate.middleware.ts`
 - **Goal:** Intercept bad data before it hits controllers.
 - **Action:** Write a generic middleware that takes a Zod schema and parses `req.body`, `req.query`, or `req.params`.
 - **How to Test:** (Will be tested in Phase 12).
-

Part 4: Authentication Layer (Level 2)

Phase 12: Auth Validators & Controllers

- **Files:** `src/validators/auth.validator.ts`, `src/controllers/auth.controller.ts`, `src/routes/auth.routes.ts`
- **Goal:** User Registration and Login endpoints.
- **Action:**
 - Zod schemas for email/password.
 - Register: Hash password with `bcrypt`, insert to DB.
 - Login: Compare hash, sign JWT using `jsonwebtoken`.
- **How to Test:** Register a user via Postman. Check Supabase to ensure the password is a hash. Login to receive the JWT.

Phase 13: JWT Protection Middleware

- **Files:** `src/middleware/auth.middleware.ts`
 - **Goal:** Lock down routes.
 - **Action:** Extract token from `Authorization: Bearer`, verify it, and attach the user record to `req.user`. Return `401 Unauthorized` if invalid.
 - **How to Test:** Hit a dummy protected route without a token (401). Hit it with the token (200).
-

Part 5: Core CRUD & Business Logic

Phase 14: Project CRUD

- **Files:** `src/validators/project.validator.ts`, `src/controllers/project.controller.ts`, `src/routes/project.routes.ts`

- **Goal:** Create, Read, Update, Delete projects.
- **Action:** Implement endpoints. Automatically set `created_by` to `req.user.id`.
- **How to Test:** Create a project in Postman. Verify it appears in Supabase with your user ID.

Phase 15: Task CRUD & Assignment Validation

- **Files:** `src/validators/task.validator.ts`,
`src/controllers/task.controller.ts`, `src/routes/task.routes.ts`
 - **Goal:** Manage tasks and ensure data integrity.
 - **Action:** Implement CRUD. **Crucial:** In the Create/Update controller, query the DB to ensure `req.body.assignedTo` actually exists in the `project_members` table for this project. Throw 400 if not.
 - **How to Test:** Try assigning a task to a user who isn't in the project. Expect a failure.
-

Part 6: Advanced Integrity & RBAC (Level 3)

Phase 16: Ownership & Membership Enforcement

- **Files:** `src/middleware/rbac.middleware.ts`
- **Goal:** Prevent unauthorized data access.
- **Action:** Write middleware ensuring a user can only `GET` a project if they are the creator OR in `project_members`. Return `403 Forbidden` otherwise.
- **How to Test:** Login as User A. Try to GET User B's private project. Expect 403.

Phase 17: Role-Based Access Control (Owner vs Member)

- **Files:** `src/middleware/rbac.middleware.ts` (Update)
- **Goal:** Differentiate permissions.
- **Action:**
 - Owner can Edit/Delete project and Tasks.
 - Member can ONLY update task status (if assigned to them).
- **How to Test:** Login as a Member. Try to change the project name (403). Try to change your task status to "Done" (200).

Phase 18: Pagination, Filtering, & Sorting

- **Files:** `src/controllers/task.controller.ts` (Update)

- **Goal:** Handle queries like `GET /tasks?status=Done&sortBy=priority&page=1&limit=10`.
 - **Action:** Parse `req.query`. Apply Drizzle `where`, `orderBy`, `limit`, and `offset`.
 - **How to Test:** Query Postman with `?status=To Do`. Ensure no "Done" tasks appear.
-

Part 7: Finalization & Polish

Phase 19: Logging, Security & Sanitization

- **Files:** `src/index.ts`, `src/utils/logger.ts`
- **Goal:** Production readiness.
- **Action:** Implement `helmet` for headers. Setup `winston` or `morgan` to log all incoming requests. Add `express-rate-limit` to `/login`.
- **How to Test:** Spam the login route in Postman to trigger a 429 Too Many Requests error. Check terminal for structured logs.

Phase 20: Documentation & Export

- **Files:** `README.md`, `CURT_Postman_Collection.json`
- **Goal:** Prepare for submission.
- **Action:** Write the README containing setup steps, Supabase configuration notes, and your API design decisions. Export the highly organized Postman collection.
- **How to Test:** Give the README to a friend (or follow it yourself in a new folder) to ensure the project runs seamlessly from scratch. Record your video walkthrough.